

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ae3ec722-12a\agent-a6867b20453665035.jsonl`
- SHA-256: `eb356cfd44a90d5a21006973d0e3fbd9a2e6bface090f0a69c4715ca340eda7a`
- Source modified: `2026-07-06T07:53:12+00:00`
- Imported at: `2026-07-08T16:00:08+00:00`
- project: `wf\_ae3ec722-12a`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T07:52:24.501Z)

Reviewing GitHub PR #37 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-c-delivery-routing -> main, one commit 100abb2).
It implements ADR 0011 Track C (docs/adr/0011-provider-neutral-domain-schema.md): populates & uses the destinations / routing_rules / delivery_attempts tables that Track B (#35, merged) created empty.
Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/delivery.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff +675/-9.

Key new code:

- db.py: `_migration_004_destinations_and_routing` (backfills destinations from `filter_config_deprecated`, `routing_rules` per source, `delivery_attempts` from `videos_deprecated.forwarded=1`); `repos` `upsert_destination`, `get_destination`, `upsert_delivery_attempt`, `record_delivery` (UPSERT with `attempt_count` increment), `get_delivery_attempt`.
- delivery.py: `forward_to_target/send_link_message/deliver` gain an optional `destination_id` kwarg; when set, `record_delivery` writes a `DeliveryAttempt`; when None (all current live callers) falls back to the `mark_forwarded` no-op.
- models.py: `Destination`, `RoutingRule`, `DeliveryStatus`, `DeliveryAttempt`.

Note: routing is NOT yet wired into the live path (`telethon_client/manager` still call `deliver()` with `destination_id=None`); `destination_id` is currently exercised only by tests. So `record_delivery` issues are LATENT until Track D, but ship now.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run a repro: C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. Build schema via `db._apply_schema(conn)` on `aiosqlite ":memory:"`; or `db.init_db(path)` on a temp file seeded with a legacy v1 schema (`users/channels/videos/filter_config`) to exercise the migration ladder (001->004). NOTE: a legacy 'videos' table MUST include columns `duration_s,width,height,size_bytes,mime_type` or migration 002's column-copy fails (that's a test-harness requirement, not a bug).
- Authoritative local gate ALREADY RUN (don't re-run full suite): `ruff check clean`; `pytest 451` passed @ 95.75% (floor 80%); `ruff format --check flags commands.py` ONLY because this branch predates the merged #36 format fix (`commands.py` is NOT in this PR) ? do NOT report that as a #37 finding.

- Report ONLY defects in THIS PR's diff. Distinguish LIVE-NOW (migration runs on upgrade) from LATENT (delivery recording, unused until Track D). No praise. Concrete failure scenario per finding. Rank severe first.

- Severity: blocker (data loss/corruption/wrong core behaviour on upgrade), high (real bug, narrow trigger), medium (latent-but-real logic bug), low (hygiene/edge). Be honest; downgrade unrealistic triggers.

DIMENSION: models.py + ADR 0011 conformance.

Investigate:

- Destination/RoutingRule/DeliveryAttempt/DeliveryStatus vs the ADR 0011 schema DDL (columns, defaults, enum values).

- SEED to verify/refute: the models.py Track-C header comment claims Track C "enforces the source/destination

invariant (a channel is either source or destination, never both)" ? is that invariant enforced ANYWHERE in

this PR (CHECK, validator, migration guard)? If not, the comment overclaims.

- Typos in comments (e.g. 'destinatinons'); frozen-model correctness; nullable FKs (route_id, policy_id).

Return findings per schema.

2. assistant (2026-07-06T07:52:26.881Z)

I'll start by reading the ADR 0011 schema and the changed models.py file to check conformance.

3. user (2026-07-06T07:52:27.923Z)

```
1      # ADR 0011: Provider-neutral domain schema (Track B)
2
3      - **Status**: Accepted (Question 1 ? Option 1A; Question 2 ? Option 2A; PR #34)
4      - **Date**: 2026-07-06
5      - **Decision owner**: project owner
6      - **Tracking issue**: #27 (Track B of the provider-neutral umbrella, #25)
7      - **Depends on**: ADR 0010 (Track A ? `accounts` + `telegram_sessions`, merged in #33)
8      - **Blocks**: #28 (Track C ? delivery/routing), #29 (Track D ? commands), and indirectly
#30 (Track E)
9
10     ## Context
11
12     ADR 0010 split `users` into `accounts` (product identity) + `telegram_sessions`
13     (MTPProto credential). That unblocked the *identity* dimension. This ADR
14     tackles the **content dimension**: how to represent sources, content items,
15     policies, destinations, routes, and delivery attempts in a provider-neutral
16     way, so that the system can ingest from Telegram today and from WhatsApp
17     (or any future provider) later without another schema rewrite.
18
19     ### What we have today (post-Track-A)
20
21     | Table | Meaning | Provider-coupling |
22     |---|---|---|
23     | `accounts` | Product identity (ADR 0010) | provider-neutral (`provider` column) |
24     | `telegram_sessions` | Optional MTPProto session (ADR 0010) | Telegram-specific by
design |
25     | `channels` | A Telegram channel/chat an account is a member of | **Telegram-specific**
(`channel_id` is a Telegram id) |
26     | `videos` | A seen Telegram video message + metadata | **Telegram-specific**
(`message_id`, `document_id` are Telegram ids) |
27     | `filter_config` | Per-account JSON blob of `FilterConfig` | provider-neutral in shape,
but scoped per-account not per-source |
28
29     The dedup logic (ADR 0006) is **3-tier and Telegram-specific**:
30     - Tier 0: `(user_id, channel_id, message_id)` ? same Telegram message.
31     - Tier 1: `(user_id, document_id)` ? same Telegram file forwarded across channels.
32     - Tier 2: `(user_id, size_bytes, duration_s, height, width)` ? re-upload fingerprint.
```

```

33
34   ### What we need
35
36   The umbrella (#25) introduces six concepts: `Source`, `ContentItem`, `Policy`,
37   `Destination`, `Route` (`routing_rules`), and `DeliveryAttempt`. This ADR
38   defines the schema for each, the two open design questions that gate the
39   schema, and the migration path from the current tables.
40
41   ## The two decisions I need from you
42
43   ### Question 1: `ContentItem.dedup_key` ? how does provider-neutral dedup relate to ADR
0006?
44
45   The current 3-tier dedup is Telegram-specific (`document_id` is a Telegram
46   concept). A provider-neutral `ContentItem` needs a dedup story that works
47   across providers. Three options:
48
49   ##### Option 1A (recommended): tiered lookup that wraps ADR 0006's tiers
50
51   Keep ADR 0006's three tiers but generalize them behind a single
52   `dedup_key` column on `content_items`:
53
54   ```
55   content_items.dedup_key TEXT NOT NULL
56   ```
57
58   The value of `dedup_key` is computed by an explicit decision tree at
59   ingestion time, provider by provider. The tree is ordered strongest-key
60   first and each branch is guarded so partial metadata can never collapse
61   to a degenerate key2 (e.g. `tg:fp:0:0:0:0`):
62
63   - Telegram:
64     1. If `document_id` is present ? `dedup_key = "tg:doc:{document_id}"`.
65     2. Else if all four fingerprint fields (`size_bytes`, `duration_s`,
66        `height`, `width`) are present ? `dedup_key = "tg:fp:{size}:{dur}:{h}:{w}"`.
67     3. Else ? `dedup_key = "tg:msg:{source_id}:{provider_message_id}"`
68        (Tier 0; the source/channel id is required because Telegram
69        `message_id` is only unique within a channel, not across sources).
70   - Future WhatsApp: `dedup_key = "wa:msg:{message_id}"` (WhatsApp message
71      ids are globally unique per account) or `"wa:sha256:{blob_hash}"` if WA
72      exposes a content hash.
73
74   The guard in step 2 (all four fields required) is what prevents the
75   partial-metadata collision the first draft of this ADR allowed. The pure
76   `compute_dedup_key(provider, *, document_id, source_id, provider_message_id,
77   size_bytes, duration_s, height, width)` function that implements this tree
78   gets its own unit tests in the implementation PR, including cases for
79   partial/null metadata.
80
81   A partial unique index on `(account_id, dedup_key) WHERE dedup_key IS NOT NULL`
82   replaces the current Tier-1 index and enforces per-account dedup at the DB
83   level, regardless of provider.
84
85   Why recommended:
86   - Zero new bandwidth ? all keys are computed from metadata already extracted
87     (honors ADR 0003's no-download principle).
88   - The provider prefix (`tg:` / `wa:`) makes cross-provider collisions
89     structurally impossible.
90   - Existing `videos` rows migrate cleanly: `dedup_key` is computed from their
91     existing `document_id` / metadata columns during the migration.
92   - Supersedes ADR 0006 cleanly: the three tiers become one column with
93     provider-specific derivation rules, documented in the migration.
94
95   Costs:
96   - The migration must compute `dedup_key` for every existing `videos` row

```

```

97     (cheap ? pure SQL string building, no I/O).
98 - The `dedup_key` derivation logic lives in code (a pure function per
99   provider), not in the schema. This is a feature, but it means the
100   derivation rules need their own unit tests.
101
102 ##### Option 1B: content hash (SHA-256 of downloaded bytes)
103
104 `dedup_key` = sha256(downloaded_content)`. Cryptographically strong,
105 provider-agnostic, catches re-encodes that 1A misses.
106
107 **Rejected** ? violates ADR 0003 (metadata-first, no download in the hot
108 path). Downloading every incoming video to hash it would multiply bandwidth
109 and latency by orders of magnitude. Could be added later as an optional
110 deep-inspect stage (deferred to Track F per #31).
111
112 ##### Option 1C: provider-assigned only (drop Tier 2)
113
114 `dedup_key` = "tg:doc:{document_id}"` always; no metadata fingerprint
115 fallback. Simpler, but loses Tier-2 dedup for re-uploads that get a new
116 `document.id` ? a real and common case (someone re-encodes and re-uploads
117 the same video to a different channel).
118
119 **Recommendation: 1A.** It preserves the dedup quality of ADR 0006, generalizes cleanly,
120 and honors the no-download constraint.
121 ---
122
123 ### Question 2: `Policy` ? existing `filter_config` ? rename, or wrap?
124
125 The codebase already has `filter_config.config_json` (one row per account,
126 holding a `FilterConfig` JSON blob ? see `models.py` and ADR 0005). The
127 umbrella introduces a `Policy` concept. They overlap. Two options:
128
129 ##### Option 2A (recommended): `Policy` *generalizes* `filter_config` (rename + extend)
130
131 `policies` is a new table; `filter_config` is migrated into it. One account
132 can own multiple policies (enables per-route filter overrides from ADR
133 0009), with a `DEFAULT` policy` per account for the no-rules-configured case.
134
135 ```
136 policies (
137     id          INTEGER PRIMARY KEY AUTOINCREMENT,
138     account_id  INTEGER NOT NULL REFERENCES accounts(id),
139     name        TEXT NOT NULL,                -- e.g. "default", "reggae-route"
140     config_json TEXT NOT NULL,                -- the FilterConfig blob (Phase 1)
141     is_default  INTEGER NOT NULL DEFAULT 0,   -- one per account (UNIQUE partial
idx)
142     created_at  TEXT NOT NULL,
143     updated_at  TEXT NOT NULL,
144     UNIQUE (account_id, name)
145 );
146 ```
147
148 **Phase 1** (this track): `config_json` holds exactly today's `FilterConfig`
149 JSON ? the rename is purely structural. **Phase 2** (Track F, deferred):
150 `config_json` grows to include topic preferences and security rules, but
151 that's behind the same column so no further migration.
152
153 Migration: for each `filter_config` row, insert a `policies` row with
154 `name='default'`, `is_default=1`, `config_json=<the existing blob>`. Drop
155 `filter_config` after the migration (keep `filter_config_deprecated` for one
156 release, mirroring ADR 0010's safety-net pattern).
157
158 **Why recommended:**
159 - `Policy` becomes the single home for "what to filter/enrich" ? no second

```

```

160     overlapping concept. Today's `FilterConfig` is just the Phase-1 shape.
161 - Enables ADR 0009's per-route filter overrides (`routing_rules.policy_id`)
162   without a second migration later.
163 - The existing `db.load_filter_config(account_id)` / `save_filter_config`
164   become thin shims over `policies WHERE account_id=? AND is_default=1`,
165   mirroring the Track-A `User` shim pattern ? backwards-compatible.
166
167 **Costs:**
168 - One more table migration. `filter_config` ? `policies` is a rename+extend.
169 - The `FilterConfig` Pydantic model stays (it's the Phase-1 config shape);
170   `Policy` is the *container* (name, owner, default flag) around it. Two
171   models, not one ? but they compose cleanly.
172
173 ##### Option 2B: keep `filter_config` as-is; `Policy` is a wrapping layer
174
175 Don't touch `filter_config`. `policies` references a `filter_config` row
176 plus adds enrichment/security stages as separate columns.
177
178 **Rejected** ? creates two overlapping concepts (`filter_config` for the
179 v1 rules, `policies` for "everything else"), which is exactly the
180 conflation the umbrella exists to dissolve. We'd be back here in a year
181 merging them.
182
183 **Recommendation: 2A.** One concept, one table, phased growth via the same JSON column.
184
185 ---
186
187 ## Proposed schema (assuming 1A + 2A)
188
189 ### `sources` (generalizes `channels`)
190
191 A configured place we ingest from. Maps 1:1 to today's per-account
192 `channels` rows for Telegram; future providers add their own source kinds.
193
194 ```
195 sources (
196     id                INTEGER PRIMARY KEY AUTOINCREMENT,
197     account_id        INTEGER NOT NULL REFERENCES accounts(id),
198     provider          TEXT NOT NULL DEFAULT 'telegram',
199     provider_source_id TEXT NOT NULL,          -- Telegram channel_id; future: WA group id
200     title             TEXT,
201     kind              TEXT NOT NULL DEFAULT 'channel', -- 'channel' | 'group' | 'feed'
202     watched           INTEGER NOT NULL DEFAULT 0,      -- ADR 0009 selective scanning
203     seen_first_at     TEXT NOT NULL,
204     last_seen_at      TEXT,
205     UNIQUE (account_id, provider, provider_source_id)
206 );
207 ```
208
209 `channels.user_id` (now ? `accounts.id`) maps to `sources.account_id`;
210 `channels.channel_id` maps to `sources.provider_source_id`. The `watched`
211 flag (ADR 0009, deferred to Track E) is included here so Track E doesn't
212 need another migration.
213
214 ### `content_items` (generalizes `videos`)
215
216 A normalized item discovered from any source. Phase 1 stores only videos
217 (matches today's `videos`); the columns are provider-neutral but the
218 Phase-1 migration only populates video fields.
219
220 ```
221 content_items (
222     id                INTEGER PRIMARY KEY AUTOINCREMENT,
223     account_id        INTEGER NOT NULL REFERENCES accounts(id),
224     source_id         INTEGER NOT NULL REFERENCES sources(id),

```

```

225     provider          TEXT NOT NULL DEFAULT 'telegram',
226     provider_message_id TEXT NOT NULL,      -- Telegram message_id; future: WA message
id
227     content_type       TEXT NOT NULL DEFAULT 'video', -- 'video' | 'link' | 'article' |
...
228     text               TEXT,                -- caption / post text (nullable)
229     link_url           TEXT,                -- for link/article posts
230     media_ref          TEXT,                -- provider media reference (file path,
blob id)
231     document_id        TEXT,                -- Telegram document.id (kept for TG-
specific lookups)
232     duration_s         INTEGER,
233     width              INTEGER,
234     height             INTEGER,
235     size_bytes         INTEGER,
236     mime_type          TEXT,
237     dedup_key          TEXT NOT NULL,        -- ADR 0011 Question 1 (Option 1A)
238     matched            INTEGER NOT NULL DEFAULT 0,
239     seen_at            TEXT NOT NULL,
240     -- Tier-0 equivalent: provider_message_id is only unique within a source
241     -- (a Telegram message_id is per-channel), so source_id is required to
242     -- avoid false collisions across sources for the same account.
243     UNIQUE (account_id, source_id, provider_message_id),
244     UNIQUE (account_id, dedup_key)          -- Tier-1/2 equivalent (Option
1A)
245 );
246 ```
247
248 **Note on `forwarded`:** dropped from `content_items`. Per-destination
249 delivery state moves to `delivery_attempts` (Track C, #28). For Phase 1
250 (before Track C lands), `content_items.matched` alone is sufficient ?
251 delivery still goes to the single `delivery_target`, and the boolean
252 `forwarded` is retained temporarily on a *deprecated* `videos`-compat view
253 so the v1 delivery path keeps working. Track C removes it.
254
255 ### `policies` (generalizes `filter_config`) ? Option 2A
256
257 As specified in Question 2 above.
258
259 ### `destinations` (new ? Track C will populate)
260
261 Defined here so the schema is complete, but **Track C (#28) implements the
262 delivery layer**. Phase-1 presence is the table only (no delivery logic).
263
264 ```
265 destinations (
266     id          INTEGER PRIMARY KEY AUTOINCREMENT,
267     account_id  INTEGER NOT NULL REFERENCES accounts(id),
268     provider    TEXT NOT NULL DEFAULT 'telegram',
269     target      TEXT NOT NULL,          -- 'me', '@username', chat id (ADR 0005)
270     kind        TEXT NOT NULL DEFAULT 'saved', -- 'saved' | 'channel' | 'chat'
271     title       TEXT,
272     is_output   INTEGER NOT NULL DEFAULT 0, -- ADR 0009: distinguishes output
channels
273     created_at  TEXT NOT NULL,
274     UNIQUE (account_id, provider, target)
275 );
276 ```
277
278 ### `routing_rules` (new ? Track C will populate)
279
280 Per ADR 0009 (canonical name locked in issue #25).
281
282 ```
283 routing_rules (

```

```

284     id            INTEGER PRIMARY KEY AUTOINCREMENT,
285     account_id    INTEGER NOT NULL REFERENCES accounts(id),
286     source_id     INTEGER NOT NULL REFERENCES sources(id),
287     policy_id     INTEGER,                -- nullable ? use account's default policy
288     destination_id INTEGER NOT NULL REFERENCES destinations(id),
289     enabled       INTEGER NOT NULL DEFAULT 1,
290     priority      INTEGER NOT NULL DEFAULT 0,
291     created_at    TEXT NOT NULL,
292     UNIQUE (account_id, source_id, destination_id)
293 );
294 ```
295
296 ### `delivery_attempts` (new ? Track C will populate)
297
298 Per #28's idempotency requirement (unique index on
299 `(content_item_id, destination_id)`).
300
301 ```
302 delivery_attempts (
303     id            INTEGER PRIMARY KEY AUTOINCREMENT,
304     content_item_id INTEGER NOT NULL REFERENCES content_items(id),
305     destination_id INTEGER NOT NULL REFERENCES destinations(id),
306     route_id      INTEGER REFERENCES routing_rules(id),
307     status        TEXT NOT NULL DEFAULT 'pending', --
'pending' | 'delivered' | 'failed' | 'retrying'
308     attempt_count INTEGER NOT NULL DEFAULT 0,
309     last_error     TEXT,
310     created_at    TEXT NOT NULL,
311     delivered_at  TEXT,
312     next_retry_at TEXT,
313     UNIQUE (content_item_id, destination_id)        -- ADR #28 idempotency
314 );
315 ```
316
317 ## Migration strategy (integer-preserving, phased)
318
319 This track lands sources, content_items, and policies only.
320 destinations, routing_rules, delivery_attempts are defined in the
321 schema (so the ADR is complete) but their tables are created empty and
322 populated by Track C. This keeps Track B focused and reviewable.
323
324 ### Migration _migration_003_domain_schema (idempotent)
325
326 1. Create sources, content_items, policies, destinations,
327 routing_rules, delivery_attempts (all IF NOT EXISTS).
328 2. Backfill sources from channels:
329 provider='telegram', provider_source_id=channel_id, watched=1
330 (ADR 0009 backfill: already-seen channels stay watched).
331 3. Backfill content_items from videos, computing dedup_key via the
332 explicit decision tree from Option 1A (every branch reachable, no
333 COALESCE(...,0) collapsing partial metadata to a degenerate key):
334 - If document_id IS NOT NULL:
335   dedup_key = 'tg:doc:' || document_id.
336 - Else if size_bytes IS NOT NULL AND duration_s IS NOT NULL AND height IS NOT NULL
AND width IS NOT NULL:
337   dedup_key = 'tg:fp:' || size_bytes || ':' || duration_s || ':' || height || ':' ||
width.
338 - Else (Tier-0 fallback; partial or missing media metadata):
339   dedup_key = 'tg:msg:' || source_id || ':' || provider_message_id,
340   where source_id is the joined sources.id for the row's legacy
341   channel_id and provider_message_id is the legacy message_id.
342
343 The implementation PR must include migration tests that seed rows hitting
344 each branch (including all-fingerprint-fields-null) and assert no two
345 rows collapse to the same dedup_key.

```

```

346 4. **Backfill `policies`** from `filter_config`: one row per account,
347     `name='default'`, `is_default=1`, `config_json` copied verbatim.
348 5. **Rename** (don't drop) `channels` ? `channels_deprecated`,
349     `videos` ? `videos_deprecated`, `filter_config` ? `filter_config_deprecated`.
350     Kept for one release as a rollback safety net (ADR 0010 pattern).
351
352 The legacy `videos`/`channels`/`filter_config` tables and their
353 `db.load_filter_config` / `upsert_channel` / `insert_video` repos stay
354 working as shims over the new tables (same pattern as the Track-A `User`
355 shim), so existing call sites in `telethon_client.py`, `backfill.py`, and
356 `commands.py` keep working without a flag day.
357
358 ## Consequences
359
360 **Positive**
361 - Provider-neutral schema: `sources`, `content_items`, `policies`,
362   `destinations`, `routing_rules`, `delivery_attempts` all carry a
363   `provider` column or provider-prefixed keys. Adding WhatsApp later is a
364   new `provider='whatsapp'` value, not a new schema.
365 - `dedup_key` (Option 1A) generalizes ADR 0006 without losing any dedup
366   quality and without downloading anything.
367 - `Policy` (Option 2A) gives a single home for filter + future enrichment
368   config, enabling ADR 0009's per-route overrides with no second migration.
369 - Backwards-compatible via shims, mirroring the Track-A pattern that
370   already passed review.
371
372 **Negative**
373 - Six new tables; the schema grows significantly. Each is small and
374   single-purpose, but the surface area is larger.
375 - The dedup-key derivation logic is in code, not the schema ? needs its
376   own unit tests (a `compute_dedup_key(provider, **metadata)` pure function).
377 - Phase 1 carries some dead-on-arrival columns (`content_items.link_url`,
378   `text`) that aren't populated until later tracks. This is intentional
379   (avoid future migrations) but means some columns are NULL until then.
380
381 ## Alternatives considered
382
383 - **Do nothing; keep `channels`/`videos`/`filter_config` and special-case
384   providers in code.** Rejected ? same conflation argument as ADR 0010.
385   Telegram-specific ids in column names (`channel_id`, `message_id`,
386   `document_id`) would leak into every future provider's code path.
387 - **Land all six tables' *logic* in this track.** Rejected ? Track B is
388   already the largest track. Splitting schema (Track B) from delivery
389   logic (Track C) keeps each PR reviewable. The empty `destinations` /
390   `routing_rules` / `delivery_attempts` tables in this track are the
391   contract Track C codes against.
392 - **`content_items` as a view over `videos` (no migration).** Rejected ?
393   views don't give us the `provider` column or the `dedup_key` index, and
394   SQLite view support for writes is limited. A real table is needed.
395
396 ## Open questions for the decision owner
397
398 1. **Question 1** ? `dedup_key`: 1A (tiered, recommended), 1B (hash,
399   rejected), or 1C (provider-assigned only).
400 2. **Question 2** ? `Policy` ? `filter_config`: 2A (generalize, recommended)
401   or 2B (wrap, rejected).
402 3. **Dead columns in Phase 1** ? acceptable to land `content_items.link_url`
403   / `text` as NULL until later tracks populate them? Recommendation: yes
404   (avoids a future `ALTER TABLE`).
405 4. **Deprecated-table retention** ? one release (default), matching ADR 0010?
406   Recommendation: yes.
407
408 ## References
409
410 - ADR 0003 ? metadata-first probing (the no-download constraint that rules

```



```

411     out Option 1B's content hashing).
412 - ADR 0005 ? per-user delivery target (generalized by `destinations` /
413   `routing_rules`).
414 - ADR 0006 ? content dedup (generalized by `content_items.dedup_key`,
415   Option 1A).
416 - ADR 0009 ? selective scanning + multiplexed routing (depends on
417   `sources.watched` and `routing_rules`, both defined here).
418 - ADR 0010 ? identity/session separation (this ADR's `account_id` FKs point
419   at `accounts.id`).
420 - Issue #27 ? Track B (this ADR's tracking issue).
421 - Issue #25 ? umbrella; defines the six concepts and the implementation order.
422

```

4. user (2026-07-06T07:52:28.006Z)

```

1     """Pydantic data models for users, channels, videos, and filter config.
2
3     These models are the in-memory representation used by the repository layer
4     (`db.py`) and the filter engine (`filters/`). They are deliberately
5     decoupled from Telethon's own types so the filter engine stays pure and
6     unit-testable without a live Telegram connection.
7     """
8
9     from __future__ import annotations
10
11     from datetime import UTC, datetime
12     from enum import Enum
13
14     from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
15
16
17     def _utcnow() -> datetime:
18         """Timezone-aware UTC 'now', used as a default factory."""
19         return datetime.now(UTC)
20
21
22     class DeliveryMode(str, Enum):
23         """How a matched video is delivered to the user.
24
25         - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
26         - ``LINK``: send a text message with a t.me link + metadata summary.
27         - ``BOTH``: forward the video AND send the link message.
28         """
29
30         FORWARD = "forward"
31         LINK = "link"
32         BOTH = "both"
33
34
35     class Account(BaseModel):
36         """A product identity ? the owner of config, channels, routes, and content.
37
38         Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39         is the bridge used by the Bot API front-door (`from.id`) and by the
40         legacy userbot path; it is nullable so future providers that don't have a
41         Telegram id (e.g. WhatsApp) can still own an ``Account``.
42         """
43
44         model_config = ConfigDict(frozen=True)
45
46         id: int = Field(description="Stable surrogate PK; the owner key for all downstream
47         tables")
48         telegram_id: int | None = Field(
49             default=None,
50             description="Telegram user id (bot from.id OR userbot account id). UNIQUE when

```

```

set.",
50         )
51         provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
52         display_name: str | None = None
53         created_at: datetime = Field(default_factory=_utcnow)
54
55
56     class TelegramSession(BaseModel):
57         """An optional MProto (Telethon) session linked to an :class:`Account`.
58
59         A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
60         that is what makes them structurally invisible to ``manager.run``
61         (ADR 0010). ``session_path`` is data-driven so the file-location
62         convention is explicit rather than derived from the PK at runtime.
63         """
64
65         model_config = ConfigDict(frozen=True)
66
67         account_id: int = Field(description="FK ? accounts.id; PK (one session per account
in v1)")
68         phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
69         session_path: str = Field(description="File-backed Telethon session path
(user_<id>.session)")
70         enabled: bool = Field(
71             default=True,
72             description="If False, the manager skips this session (not the account)",
73         )
74         created_at: datetime = Field(default_factory=_utcnow)
75         last_login_at: datetime | None = None
76
77
78     class User(BaseModel):
79         """Deprecated: legacy v1 model conflating identity + MProto session.
80
81         Retained as a backwards-compatible view over the new ``accounts`` +
82         ``telegram_sessions`` tables (ADR 0010). New code should use
83         :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
84         maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
85         and tests keep working without a flag day.
86         """
87
88         model_config = ConfigDict(frozen=True)
89
90         id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy
rows)")
91         phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
92         enabled: bool = Field(default=True, description="If False, the manager skips this
session")
93         created_at: datetime = Field(default_factory=_utcnow)
94
95
96     class Channel(BaseModel):
97         """A channel/group a user is a member of. Populated lazily as messages arrive.
98
99         Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
100         code should use :class:`Source` directly; the ``db`` layer maps
101         ``Channel`` ? ``Source`` so existing callers and tests keep working.
102         """
103
104         model_config = ConfigDict(frozen=True)
105
106         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")

```

```

107     user_id: int
108     channel_id: int = Field(description="Telegram channel/chat id")
109     title: str | None = None
110     seen_first_at: datetime = Field(default_factory=_utcnow)
111
112
113 # --- provider-neutral domain models (ADR 0011, Track B) -----
114 #
115 # These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig``
116 # shapes so the schema no longer leaks Telegram-specific ids into every future
117 # provider's code path. ``provider`` columns + provider-prefixed ``dedup_key``
118 # make cross-provider collisions structurally impossible. See ADR 0011 for the
119 # full rationale and the two decisions (Option 1A: tiered ``dedup_key``;
120 # Option 2A: ``policies`` generalizes ``filter_config``).
121
122
123 class Source(BaseModel):
124     """A configured place we ingest content from (ADR 0011).
125
126     Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127     providers add their own source kinds). ``provider_source_id`` is the
128     provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129     """
130
131     model_config = ConfigDict(frozen=True)
132
133     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134     account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135     provider: str = Field(default="telegram", description="Provider namespace")
136     provider_source_id: str = Field(
137         description="Provider-native source id (Telegram channel_id, ?)"
138     )
139     title: str | None = None
140     kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141     watched: bool = Field(
142         default=False,
143         description="ADR 0009 selective scanning: ingest from this source iff True",
144     )
145     seen_first_at: datetime = Field(default_factory=_utcnow)
146     last_seen_at: datetime | None = None
147
148
149 class ContentItem(BaseModel):
150     """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152     Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153     (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154     NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155     ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156     wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157     """
158
159     model_config = ConfigDict(frozen=True)
160
161     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162     account_id: int = Field(description="FK ? accounts.id")
163     source_id: int = Field(description="FK ? sources.id")
164     provider: str = Field(default="telegram", description="Provider namespace")
165     provider_message_id: str = Field(
166         description="Provider-native message id (Telegram message_id, ?)"
167     )
168     content_type: str = Field(
169         default="video", description="'video' | 'link' | 'article'"
170     )
171     text: str | None = Field(
172         default=None, description="Caption / post text (nullable in Phase 1)"
173     )

```

```

170         link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171         media_ref: str | None = Field(default=None, description="Provider media reference")
172         document_id: str | None = Field(
173             default=None,
174             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175         )
176         duration_s: int | None = None
177         width: int | None = None
178         height: int | None = None
179         size_bytes: int | None = None
180         mime_type: str | None = None
181         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182         matched: bool = False
183         seen_at: datetime = Field(default_factory=_utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same ``config_json`` column.
192         Exactly one policy per account carries ``is_default=True``; the legacy
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197
198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199         account_id: int = Field(description="FK ? accounts.id")
200         name: str = Field(description="Policy name; 'default' for the account default
policy")
201         config_json: str = Field(
202             description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203         )
204         is_default: bool = Field(
205             default=False, description="Exactly one per account (UNIQUE partial index)"
206         )
207         created_at: datetime = Field(default_factory=_utcnow)
208         updated_at: datetime = Field(default_factory=_utcnow)
209
210
211     # --- Track C: destinations, routing, delivery state (ADR 0011) -----
212     #
213     # These tables were created empty by Track B (_SCHEMA); Track C populates
214     # destinatinons (backfilled from filter_config.delivery_target), writes
215     # delivery_attempts (replacing the legacy videos.forwarded boolean), and
216     # enforces the source/destination invariant (a channel is either source or
217     # destination, never both).
218
219
220     class Destination(BaseModel):
221         """A delivery target ? where matched content is sent (ADR 0011, Track C).
222
223         Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
224         can have multiple named output channels. Backfilled from the existing
225         per-account ``delivery_target``; the default destination is the target used
226         when no explicit routing rule matches.
227         """
228
229         model_config = ConfigDict(frozen=True)
230
231         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")

```

```

232     account_id: int = Field(description="FK ? accounts.id")
233     provider: str = Field(default="telegram", description="Provider namespace")
234     target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
235     kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
236     title: str | None = None
237     is_output: bool = Field(
238         default=False,
239         description="ADR 0009: distinguishes output channels from ingest sources",
240     )
241     created_at: datetime = Field(default_factory=_utcnow)
242
243
244 class RoutingRule(BaseModel):
245     """A rule that maps a source through a policy to a destination (ADR 0009).
246
247     Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
248     ``policy_id`` is NULL, the account's default policy is used. When multiple
249     rules match, the highest ``priority`` wins.
250     """
251
252     model_config = ConfigDict(frozen=True)
253
254     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
255     account_id: int = Field(description="FK ? accounts.id")
256     source_id: int = Field(description="FK ? sources.id")
257     policy_id: int | None = Field(
258         default=None,
259         description="FK ? policies.id; NULL means use account default policy",
260     )
261     destination_id: int = Field(description="FK ? destinations.id")
262     enabled: bool = Field(default=True)
263     priority: int = Field(default=0, description="Higher wins when multiple rules
match")
264     created_at: datetime = Field(default_factory=_utcnow)
265
266
267 class DeliveryStatus(str, Enum):
268     """Per-destination delivery state, replacing the legacy binary ``forwarded``."""
269
270     PENDING = "pending"
271     DELIVERED = "delivered"
272     FAILED = "failed"
273     RETRYING = "retrying"
274
275
276 class DeliveryAttempt(BaseModel):
277     """Per-destination delivery record ? idempotent via UNIQUE index.
278
279     Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
280     ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
281     incremented and ``status``/``last_error`` are updated.
282     """
283
284     model_config = ConfigDict(frozen=True)
285
286     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
287     content_item_id: int = Field(description="FK ? content_items.id")
288     destination_id: int = Field(description="FK ? destinations.id")
289     route_id: int | None = Field(default=None, description="FK ? routing_rules.id
(nullable)")
290     status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
291     attempt_count: int = Field(default=0, description="Incremented on each retry")
292     last_error: str | None = None
293     created_at: datetime = Field(default_factory=_utcnow)
294     delivered_at: datetime | None = None

```

```

295     next_retry_at: datetime | None = None
296
297
298 class FilterConfig(BaseModel):
299     """Per-user filter configuration.
300
301     v1 rules: duration bounds, minimum height (resolution), and max file size.
302     Stored in the DB as a JSON blob (`filter_config.config_json`) so adding a
303     new field later is backwards-compatible.
304     """
305
306     # Duration bounds in seconds. None means unbounded on that side.
307     min_duration_s: int | None = Field(default=5, ge=0)
308     max_duration_s: int | None = Field(default=7200, ge=0)
309
310     # Resolution: a video passes if its height >= target_height.
311     # None disables the resolution rule.
312     target_height: int | None = Field(default=1080, ge=1)
313
314     # Max file size in mebibytes. None disables the size rule.
315     max_size_mb: int | None = Field(default=2048, ge=1)
316
317     # Content-type exclusions. GIFs and round videos are excluded by default
318     # (their duration/dimensions are usually incidental to real video filters);
319     # silent videos are off by default since some legit clips have no audio.
320     # Existing users' stored configs are unaffected ? the DB JSON blob overrides
321     # these defaults, so this only changes the behaviour for fresh installs.
322     exclude_gif: bool = Field(
323         default=True,
324         description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
325     )
326     exclude_round: bool = Field(
327         default=True,
328         description="If True, exclude circular video messages (round_message=True)",
329     )
330     exclude_nosound: bool = Field(
331         default=False,
332         description="If True, exclude videos without an audio track (nosound=True)",
333     )
334
335     # How matched videos are delivered: forward the video, send a link, or both.
336     # Stored in the same JSON blob, so adding this field is backwards-compatible
337     # (existing rows without it default to FORWARD).
338     delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
339
340     # Where matched videos / link messages are sent. Accepts:
341     # - "me" (default): Saved Messages ? exact v1.1 behaviour.
342     # - "@username": a public channel the account is a member of.
343     # - "-100..." or a bare int-as-string: a numeric Telegram chat id
344     #   (private channels without a username).
345     # The string is passed verbatim to Telethon's forward_messages / send_message.
346     # See ADR 0005 for rationale and access requirements.
347     delivery_target: str = Field(
348         default="me",
349         min_length=1,
350         description="Telethon entity to deliver to: 'me', '@username', or a chat id",
351     )
352
353     @field_validator("delivery_target", mode="before")
354     @classmethod
355     def _coerce_delivery_target(cls, v: object) -> str:
356         """Accept int chat ids (e.g. -1001234567890) and coerce to str.
357
358         Telethon accepts both str and int as an entity, but storing as str keeps
359         the JSON blob uniform and lets '@username' / 'me' / numeric ids share

```

```

360         one field type.
361         """
362         if v is None:
363             return "me"
364         if isinstance(v, int):
365             return str(v)
366         return str(v)
367
368     @model_validator(mode="after")
369     def _check_duration_bounds(self) -> FilterConfig:
370         """Ensure min_duration_s <= max_duration_s when both are set."""
371         if (
372             self.min_duration_s is not None
373             and self.max_duration_s is not None
374             and self.min_duration_s > self.max_duration_s
375         ):
376             raise ValueError(
377                 f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s
378                 ({self.max_duration_s})"
379             )
380         return self
381
382     class VideoRecord(BaseModel):
383         """A seen video message + its metadata. Drives dedup and future /search.
384
385         Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
386         New code should use :class:`ContentItem`; the ``db`` layer maps
387         ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
388         """
389
390         model_config = ConfigDict(frozen=True)
391
392         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
393         user_id: int
394         channel_id: int = Field(description="Origin Telegram channel id")
395         message_id: int = Field(description="Telegram message id within the channel")
396         document_id: int | None = Field(
397             default=None,
398             description="Telegram document.id ? reused across forwards; Tier-1 content
399 dedup",
400         )
401
402         # Metadata extracted from the Telethon message (may be None if absent).
403         duration_s: int | None = None
404         width: int | None = None
405         height: int | None = None
406         size_bytes: int | None = None
407         mime_type: str | None = None
408
409         matched: bool = False
410         forwarded: bool = False
411         seen_at: datetime = Field(default_factory=_utcnow)
412
413     @property
414     def dedup_key(self) -> tuple[int, int, int]:
415         """The natural key used for deduplication in the ``videos`` table."""
416         return (self.user_id, self.channel_id, self.message_id)
417
418     @property
419     def content_fingerprint(self) -> tuple[int, int, int, int] | None:
420         """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height,
421 width).
422
423         Returns None if any field is missing (cannot fingerprint).

```

```

422         """
423         if None in (self.size_bytes, self.duration_s, self.height, self.width):
424             return None
425         return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]
426
427
428     # Default filter config shared by the bootstrap onboarding step.
429     DEFAULT_FILTER_CONFIG = FilterConfig()
430

```

5. assistant (2026-07-06T07:52:35.989Z)

Now let me check the db.py migration and delivery.py to verify the invariant claim and the record_delivery logic.

6. user (2026-07-06T07:52:38.136Z)

```

src\tg_video_filter\models.py:214:# destinatinons (backfilled from
filter_config.delivery_target), writes
src\tg_video_filter\models.py:216:# enforces the source/destination invariant (a channel is
either source or
src\tg_video_filter\models.py:217:# destination, never both).

```

7. user (2026-07-06T07:52:38.893Z)

```

668:async def _migration_004_destinations_and_routing(conn: aiomysql.Connection) -> None:

```

8. assistant (2026-07-06T07:52:41.834Z)

The invariant is only mentioned in the models.py comment ? let me confirm it's not enforced anywhere in db.py migration, and check the schema DDL for the four Track-C tables to compare against the models.

9. user (2026-07-06T07:52:42.946Z)

```

668     async def _migration_004_destinations_and_routing(conn: aiomysql.Connection) -> None:
669         """Backfill default destinations and delivery state (ADR 0011, Track C).
670
671         Creates a default destination row for each account that has a
672         ``filter_config_deprecated.delivery_target`` value, then creates a default
673         routing rule (source ? default policy ? destination) for each source.
674         Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
675         table (one ``delivered`` attempt per previously-forwarded video to the
676         account's default destination).
677
678         Idempotent: skips accounts whose default destination already exists.
679         """
680         # Gate: if destinations already have data, this migration has run.
681         async with conn.execute("SELECT COUNT(*) FROM destinations") as cur:
682             if (await cur.fetchone())[0] > 0:
683                 return
684
685         if not await _table_exists(conn, "filter_config_deprecated"):
686             return
687
688         # 1. Backfill default destinations from filter_config_deprecated.
689         #     Parse the JSON delivery_target, defaulting to "me".
690         await conn.execute(
691             """
692             INSERT OR IGNORE INTO destinations (account_id, provider, target, kind,
is_output, created_at)
693             SELECT user_id, 'telegram',
694                    COALESCE(json_extract(config_json, '$.delivery_target'), 'me'),

```



```

695         CASE
696             WHEN COALESCE(json_extract(config_json, '$.delivery_target'), 'me') =
'me'
697                 THEN 'saved'
698                 ELSE 'channel'
699             END,
700             0,
701             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
702         FROM filter_config_deprecated
703         """)
704     )
705
706     # 2. Create a default routing rule per source (source ? default policy ? default
destination).
707     await conn.execute(
708         """
709         INSERT OR IGNORE INTO routing_rules
710             (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
711         SELECT s.account_id, s.id, p.id, d.id, 1, 0,
712             strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
713         FROM sources s
714         JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
715         JOIN destinations d ON d.account_id = s.account_id
716         WHERE d.target = (
717             SELECT COALESCE(
718                 json_extract(fcd.config_json, '$.delivery_target'), 'me'
719             )
720         FROM filter_config_deprecated fcd
721         WHERE fcd.user_id = s.account_id
722         LIMIT 1
723         )
724         """)
725     )
726
727     # 3. Backfill delivery_attempts from previously-forwarded videos.
728     await conn.execute(
729         """
730         INSERT OR IGNORE INTO delivery_attempts
731             (content_item_id, destination_id, route_id, status, attempt_count,
732             created_at, delivered_at)
733         SELECT ci.id, d.id, rr.id, 'delivered', 1,
734             vd.seen_at, vd.seen_at
735         FROM videos_deprecated vd
736         JOIN content_items ci
737             ON ci.account_id = vd.user_id
738             AND ci.provider = 'telegram'
739             AND ci.provider_message_id = CAST(vd.message_id AS TEXT)
740         JOIN sources sc
741             ON sc.id = ci.source_id
742             AND sc.provider_source_id = CAST(vd.channel_id AS TEXT)
743         JOIN destinations d ON d.account_id = vd.user_id
744         JOIN routing_rules rr
745             ON rr.account_id = vd.user_id
746             AND rr.source_id = sc.id
747             AND rr.destination_id = d.id
748         WHERE vd.forwarded = 1
749         """)
750     )
751
752
753     async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
754         """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
755         async with conn.execute(
756             "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,))

```

```

757         ) as cur:
758             return await cur.fetchone() is not None
759
760
761 # --- helpers -----
762
763
764 def _now() -> datetime:
765     """Timezone-aware UTC 'now', used for policy timestamps."""
766     return datetime.now(UTC)
767
768
769 def _safe_int_cast(value: object, field_name: str) -> int:
770     """Cast a provider_source_id / provider_message_id / document_id to int.
771
772     All existing data is Telegram ? these ids are numeric strings. Non-Telegram
773     providers cannot be projected onto the legacy ``Channel`` / ``VideoRecord``
774     shapes (which carry Telegram-specific int ids); this function raises a clear
775     ``ValueError`` rather than letting a cryptic ``int()`` crash propagate
776     (ADR 0011 provider-neutrality guard ? review finding #5).
777     """
778     try:
779         return int(value) # type: ignore[arg-type]
780     except (ValueError, TypeError) as exc:
781         raise ValueError(
782             f"Cannot cast {field_name}={value!r} to int. The legacy Channel/VideoRecord
783             "
784             f"types are Telegram-specific; use ContentItem/Source directly for "
785             f"non-Telegram providers (ADR 0011)."
786         ) from exc
787

```

10. user (2026-07-06T07:52:44.166Z)

```

143:CREATE TABLE IF NOT EXISTS destinations (
155:CREATE TABLE IF NOT EXISTS routing_rules (
167:CREATE TABLE IF NOT EXISTS delivery_attempts (

```

11. assistant (2026-07-06T07:52:47.149Z)

The invariant is confirmed NOT enforced anywhere ? it's only in a comment. Now let me read the schema DDL for the three tables to compare column-by-column with the models, since this is the conformance dimension.

12. user (2026-07-06T07:52:47.954Z)

```

143 CREATE TABLE IF NOT EXISTS destinations (
144     id            INTEGER PRIMARY KEY AUTOINCREMENT,
145     account_id    INTEGER NOT NULL REFERENCES accounts(id),
146     provider      TEXT NOT NULL DEFAULT 'telegram',
147     target        TEXT NOT NULL,
148     kind          TEXT NOT NULL DEFAULT 'saved',
149     title         TEXT,
150     is_output     INTEGER NOT NULL DEFAULT 0,
151     created_at    TEXT NOT NULL,
152     UNIQUE (account_id, provider, target)
153 );
154
155 CREATE TABLE IF NOT EXISTS routing_rules (
156     id            INTEGER PRIMARY KEY AUTOINCREMENT,
157     account_id    INTEGER NOT NULL REFERENCES accounts(id),
158     source_id     INTEGER NOT NULL REFERENCES sources(id),
159     policy_id     INTEGER REFERENCES policies(id),

```

```

160     destination_id INTEGER NOT NULL REFERENCES destinations(id),
161     enabled         INTEGER NOT NULL DEFAULT 1,
162     priority        INTEGER NOT NULL DEFAULT 0,
163     created_at      TEXT NOT NULL,
164     UNIQUE (account_id, source_id, destination_id)
165 );
166
167 CREATE TABLE IF NOT EXISTS delivery_attempts (
168     id                INTEGER PRIMARY KEY AUTOINCREMENT,
169     content_item_id   INTEGER NOT NULL REFERENCES content_items(id),
170     destination_id    INTEGER NOT NULL REFERENCES destinations(id),
171     route_id         INTEGER REFERENCES routing_rules(id),
172     status            TEXT NOT NULL DEFAULT 'pending',
173     attempt_count     INTEGER NOT NULL DEFAULT 0,
174     last_error        TEXT,
175     created_at        TEXT NOT NULL,
176     delivered_at      TEXT,
177     next_retry_at     TEXT,
178     UNIQUE (content_item_id, destination_id)
179 );
180 ""
181
182 # Note: the ``idx_videos_user_document`` partial unique index is created by
183 # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
184 # which is present on fresh tables but only added to legacy v1 DBs by the
185 # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
186 # DBs that don't yet have the column.
187

```

13. assistant (2026-07-06T07:52:55.860Z)

The DDL matches the models and ADR exactly for all three tables. The models conform: Destination (no status/enum needed), RoutingRule (policy_id nullable, destination_id NOT NULL), DeliveryAttempt (route_id nullable, status enum matches the four values pending/delivered/failed/retrying). All defaults match.

Now let me verify my two main findings. Finding 1: the invariant overclaim. Finding 2: the typo 'destinatinons'. Both are in the models.py comment block (lines 211-217). Let me confirm there's no CHECK or validator enforcing it anywhere in the diff by checking the migration doesn't add such a guard (confirmed above ? migration 004 has no invariant check).

Let me quickly verify enum default conformance and that DeliveryStatus values match the DDL exactly, plus check whether `is\_output=0` backfill contradicts anything. The models are frozen (ConfigDict(frozen=True)) on all four ? consistent with the rest of the file.

The models.py dimension is clean on schema conformance. The two defects are both in the comment block. Let me record findings.

14. user (2026-07-06T07:53:11.923Z)

Structured output provided successfully